

Chapter 4

CONTROL STRUCTURES

Outline

- **1. Selection criteria**
- **2. The *if-else* statement**
- **3. Nested *if* statement**
- **4. Repetition structure**
- **5. *while* loops**
- **6. *for* loops**
- **7. Nested loops**
- **8. do-while Loops**

1. Selection structure

- The flow of control means the order in which a program's statements are executed.
- Unless directed otherwise, the normal flow of control for all programs is *sequential*.
- Selection, repetition and function invocation structures permit the flow of control to be *altered* in a defined way.
- In this chapter, you learn to use selection structures and repetition structures in C++

SELECTION CRITERIA

■ Comparison Operators

Comparison operators are used to compare two operands for equality or to determine if one numeric value is greater than another.

A Boolean value of *true* or *false* is returned after two operands are compared. C++ uses a *nonzero* value to represent a *true* and a *zero* value to represent a *false* value.

Operator	Description	Examples
==	equal	<code>a == 'y'</code>
!=	not equal	<code>m != 5</code>
>	greater than	<code>a*b > 7</code>
<	less than	<code>b < 6</code>
<=	less than or equal	<code>b <= a</code>
>=	greater than or equal	<code>c >= 6</code>

Logical operators

- Logical operators are used for creating more complex conditions. Like comparison operators, a Boolean value of *true* or *false* is returned after the logical operation is executed.

Operator	Description

&&	AND
	OR
!	NOT

- **Example:**

(age > 40) && (term < 10)

(age > 40) || (term < 10)

!(age > 40)

(i==j) || (a < b) || complete

Operator precedence

- The relational and logical operators have a hierarchy of execution similar to the arithmetic operators.

Level	Operator	Associativity

1.	! unary - ++ --	Right to left
2.	* / %	Left to right
3.	+ -	Left to right
4.	< <= > >=	Left to right
5.	= = !=	Left to right
6.	&&	Left to right
7.		Left to right
8.	= += -= *= /=	Right to left

- **Example: Assume the following declarations:**

char key = 'm';

int i = 5, j = 7, k = 12;

double x = 22.5;

Expression	Equivalent	Value	Interpretation

i + 2 == k-1	(i + 2) == (k -1)	0	false
'a' +1 == 'b'	('a' +1) == 'b'	1	true
25 >= x + 1.0	25 >= (x + 1.0)	1	true
key -1 > 20	(key -1) > 20	0	false

Order of evaluation

The following compound condition is evaluated as:

$(6*3 == 36/2) \parallel (13 < 3*3 + 4) \&\& !(6-2 < 5)$

$(18 == 18) \parallel (13 < 9 + 4) \&\& !(4 < 5)$

$1 \parallel (13 < 13) \&\& !1$

$1 \parallel 0 \&\& 0$

$1 \parallel 0$

1

The *bool* Data Type

- As specified by the ANSO/ISO standard, C++ has a built-in Boolean data type, *bool*, containing the two values *true* and *false*.
- The actual values represented by the bool values, *true* and *false*, are the integer values 1 and 0, respectively.

■ Example 4.1.1

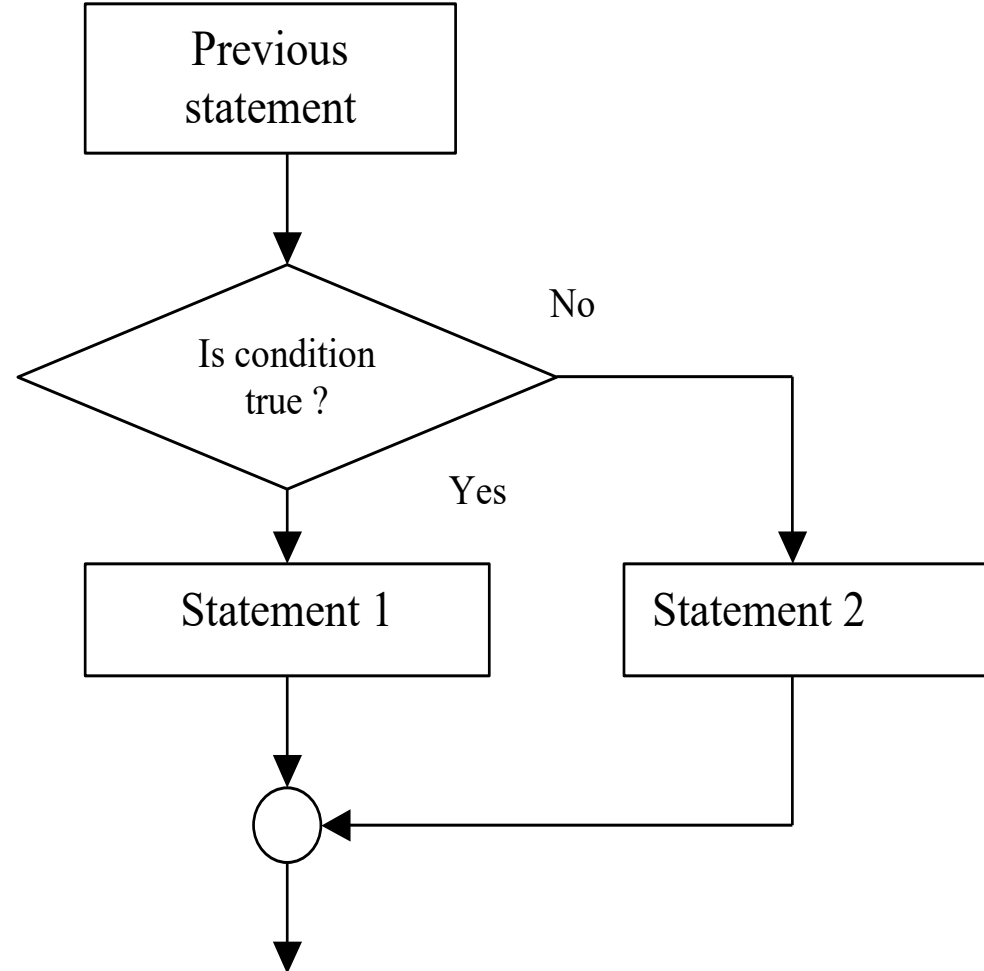
```
#include<iostream>
using namespace std;
int main()
{
    bool t1, t2;
    t1 = true;
    t2 = true;
    cout << "The value of t1 is "<< t1
        << "\n and the value of t2 is "<< t2 << endl;
    return 0;
}
```

2. THE if-else STATEMENT

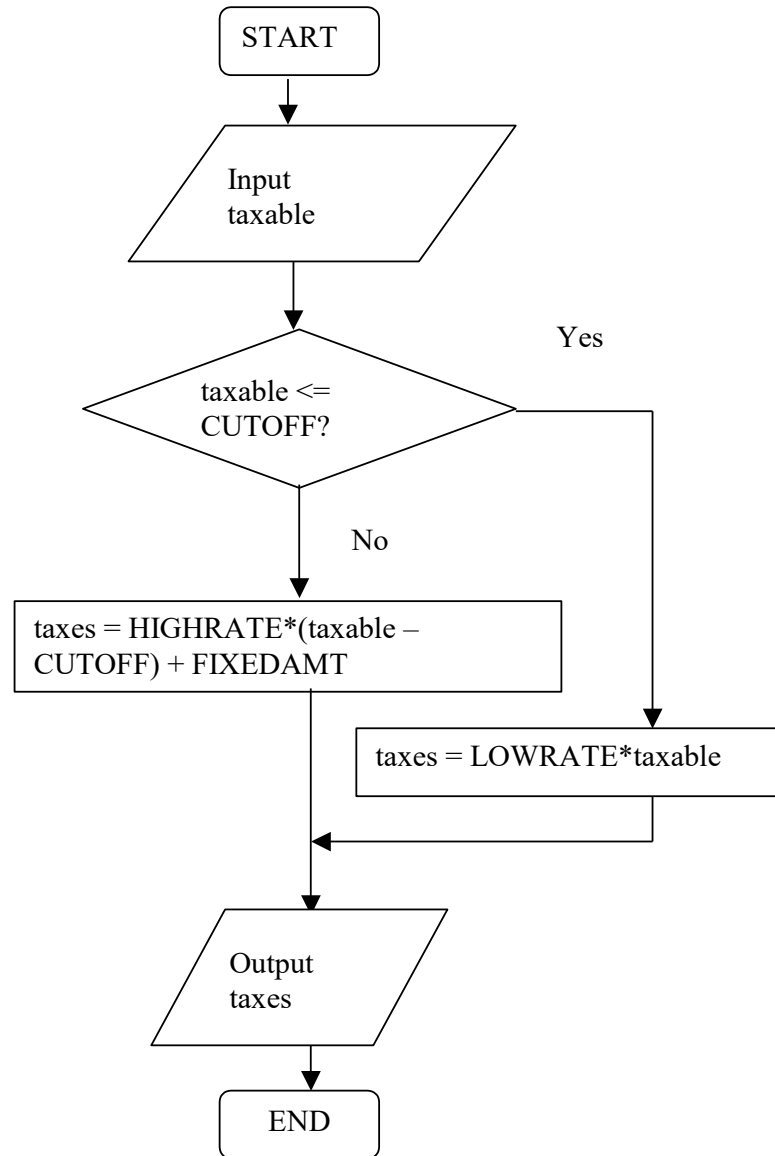
The *if-else* statement directs the computer to select a sequence of one or more statements based on the result of a comparison.

The syntax:

```
if (conditional expression) {  
    statements;  
}  
else {  
    statements;  
}
```



//if only 1 statement, then we can skip the block {} for if-else



Example 4.2.1

We construct a C++ program for determining income taxes.

Assume that these taxes are assessed at 2% of taxable incomes less than or equal to \$20,000. For taxable income greater than \$20,000, taxes are 2.5% of the income that exceeds \$20,000 plus a fixed amount of \$400.

Example 4.2.1

```
#include <iostream>
#include <iomanip>
using namespace std;
const float LOWRATE = 0.02;    // lower tax rate
const float HIGHRATE = 0.025;  // higher tax rate
const float CUTOFF = 20000.0;  // cut off for low rate
const float FIXEDAMT = 400;
int main()
{
    float taxable, taxes;
    cout << "Please type in the taxable income: ";
    cin >> taxable;
    if (taxable <= CUTOFF)
        taxes = LOWRATE * taxable;
    else
        taxes = HIGHRATE * (taxable - CUTOFF) + FIXEDAMT;
    // set output format
    cout << setiosflags(ios::fixed)
         << setiosflags(ios::showpoint)
         << setprecision(2);
    cout << "Taxes are $ " << taxes << endl;
    return 0;
}
```

setiosflags this manipulator is used to control different input and output settings.

setioflag(ios::fixed) means the output field will use conventional fixed-point decimal notation. (ex: 12.568)

setiosflag(ios::showpoint) means the output field will show the decimal point for floating point number.

setiosflag(ios::scientific) means the output field will use exponential notation. (1.0e-10)

The results of the above program:

Please type in the taxable income: 10000

Taxes are \$ 200

and

Please type in the taxable income: 30000

Taxes are \$ 650

Block Scope

- All statements within a compound statement constitute a single **block** of code, and any variable declared within such a block only is valid within the block.
- The location within a program where a variable can be used formally referred to as the **scope** of the variable.

■ Example:

```
{ // start of outer block
    int a = 25;
    int b = 17;
    cout << "The value of a is " << a << " and b is " << b << endl;
    { // start of inner block
        float a = 46.25;
        int c = 10;
        cout << " a is now " << a << "b is now " << b
            << " and c is " << c << endl;
    }
    cout << " a is now " << a
        << "b is now " << b << endl;
} // end of outer block
```

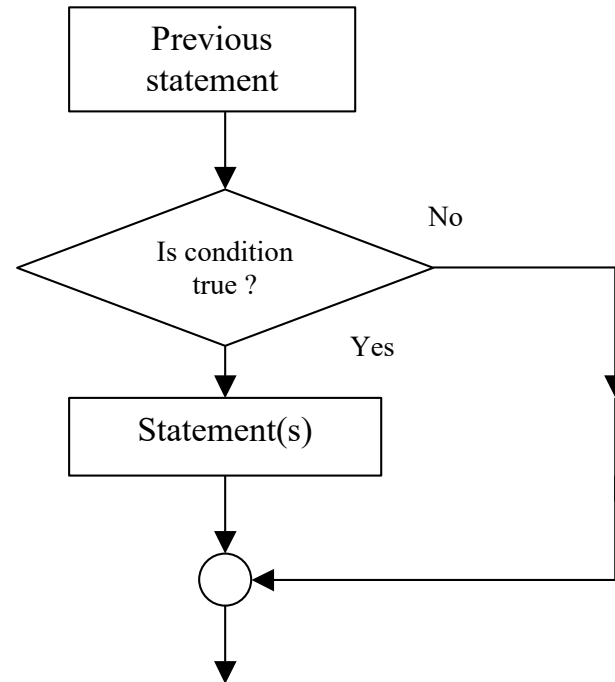
The output is

```
The value of a is 25  and b is 17
a is now 46.25 b is now 17 and c is 10
a is now 25 b is now 17
```

One-way Selection

- A useful modification of the *if-else* statement involves omitting the *else* part of the statement. In this case, the *if* statement takes a shortened format:

```
if (conditional expression) {  
    statements;  
}
```



Example 4.2.2

The following program displays an error message for the grades that is less than 0 or more than 100.

```
#include <iostream>
using namespace std;
int main()
{
    int grade = 0;
    cout << "\nPlease enter a grade: ";
    cin >> grade;
    if(grade < 0 || grade > 100)
        cout << " The grade is not valid\n";
    return 0;
}
```

3. NESTED if STATEMENT

- The inclusion of one or more *if* statement within an existing if statement is called a *nested if statement*.
- The if-else Chain

When an *if* statement is included in the *else* part of an existing *if* statement, we have an *if-else chain*.

```
if (expression-1)  
    statement-1  
else if (expression-2)  
    statement-2  
else  
    statement-3
```

- Example 4.3.1

// This program can solve quadratic equation

```

#include <iostream>
#include <cmath>
#include <iomanip>
using namespace std;
int main()
{
    double a, b, c, del, x1, x2;
    cout << "Enter the coefficients of the equation: " << endl;
    cin >> a >> b >> c;
    del = b*b - 4.0*a*c;
    if (del == 0.0)
    {
        x1 = x2 = -b/(2*a);
        cout << "x1 = " << x1 << setw(20) << "x2 = " << x2 << endl;
    }
    else if (del > 0.0)
    {
        x1 = (-b + sqrt(del))/(2*a);
        x2 = (-b - sqrt(del))/(2*a);
        cout << "x1 = " << x1 << setw(20) << "x2 = " << x2 << endl;
    }
    else // del < 0
        cout << "There is no solution\n";
    return 0;
}

```

The output of the above program:

Enter the coefficients of the equation:

1 5 6
x1 = -2.0 x2 = -3.0

4. Repetition structures

- **C++ provides three different forms of repetition structures:**
 1. *while* structure
 2. *for* structure
 3. *do-while* structure
- **Each of these structures requires a condition that must be evaluated.**
- **The condition can be tested at either (1) the beginning or (2) the end of the repeating section of code.**
- **If the test is at the beginning of the loop, the type of loop is a *pre-test loop*.**
- **If the test is at the end of the loop, the type of loop is a *post-test loop*.**

Fixed count loop and variable condition loop

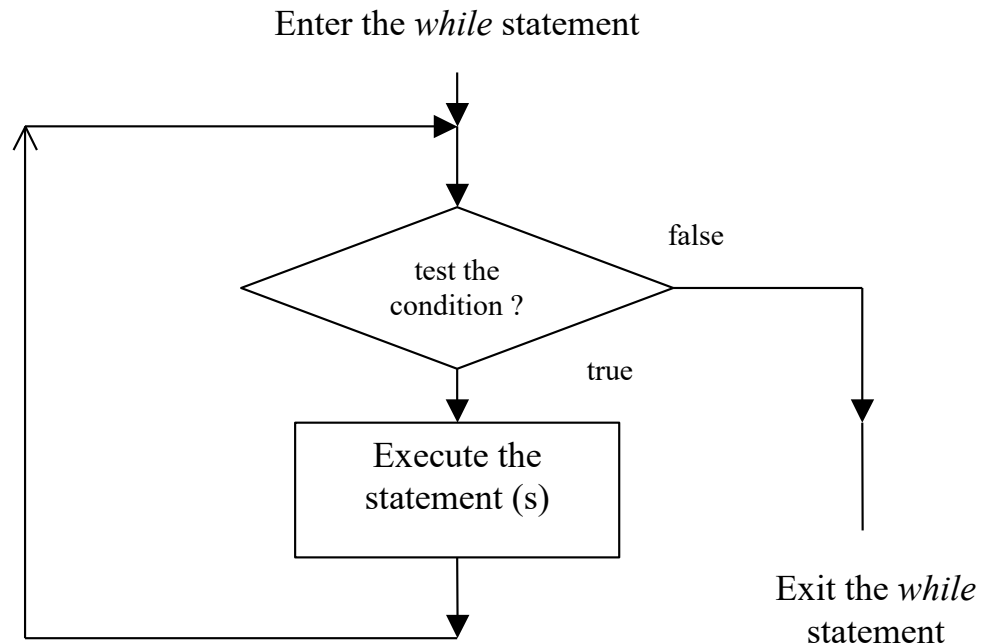
- In addition to where the condition is tested, repeating sections of code are also classified.
- In a *fixed count loop*, the condition is used to keep track of how many repetitions have occurred. In this kind of loops, a fixed number of repetitions are performed, at which point the repeating section of code is exited.
- In many situations, the exact number of repetitions are not known in advance. In such cases, a *variable condition loop* is used.
- In a *variable condition loop*, the tested condition does not depend on a count being achieved, but rather on a variable that can change interactively with each pass through the loop. When a specified value is encountered, regardless of how many iterations have occurred, repetitions stop.

5. *while* loops

The *while* statement is used for repeating a statement or series of statements as long as a given conditional expression is evaluated to true.

The syntax for the *while* statement:

```
while (condition expression) {  
    statements;  
}
```



Example 5.2.1

// This program prints out the numbers from 1 to 10

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int count;
```

```
    count = 1;           // initialize count
```

```
    while (count <= 10) {
```

```
        cout << count << " ";
```

```
        count++;         // increment count
```

```
    }
```

```
    return 0;
```

```
}
```

The output of the above program:

1 2 3 4 5 6 7 8 9 10

In the above program, the loop incurs a ***counter-controlled repetition***. Counter-controlled repetition requires:

- 1) the name of a control variable (the variable *count*)
- 2) the initial value of the control variable (*count* is initialized to 1 in this case)
- 3) the condition that tests for the final value of the control variable (i.e., whether looping should continue) ;
- 4) the *increment* (or *decrement*) by which the control variable is modified each time through the loop.

Sentinels

- In programming, data values used to indicate either the start or end of a data series are called *sentinels*.
- The sentinels must be selected so as not to conflict with legitimate data values.

Example 5.3.2

```
#include <iostream>
using namespace std;
const int HIGHGRADE = 100; // sentinel value
int main()
{
    float grade, total;
    grade = 0;
    total = 0;
    cout << "\nTo stop entering grades, type in any number"
         << " greater than 100.\n\n";
```

```
cout << "Enter a grade: ";
cin >> grade;
while (grade <= HIGHGRADE)
{
    total = total + grade;
    cout << "Enter a grade: ";
    cin >> grade;
}
cout << "\nThe total of the grades is " << total << endl;
return 0;
}
```

- In the above program, the *sentinel* is the value 100 for the entered grade.

break statement

- The *break* statement causes an exit from the innermost enclosing loop.

Example:

```
while (count <= 10)
{
    cout << "Enter a number: ";    cin >> num;
    if (num > 76) {
        cout << "you lose!\n";
        break;
    }
    else
        cout << "Keep on trucking!\n";
    count++;
}
```

//break jumps to here

continue Statements

- The *continue* statement halts a looping statement and restarts the loop with a new iteration.

```
int count = 0;
while (count < 30) {
    cout << "Enter a grade: ";
    cin >> grade;
    if (grade < 0 || grade > 100)
        continue;
    total = total + grade;
    count++;
}
```

- In the above program, invalid grades are simply ignored and only valid grades are added to the total.

The null statement

- All statements must be terminated by a semicolon. A semicolon with nothing preceding it is also a valid statement, called the *null statement*. Thus, the statement

;

is a null statement.

Example:

```
if (a > 0)
    b = 7;
else ;
```

goto statement

A *goto* statement is a kind of jump statement. Its destination is specified by a label within the statement. A label is simply an identifier followed by a statement, separated by a colon.

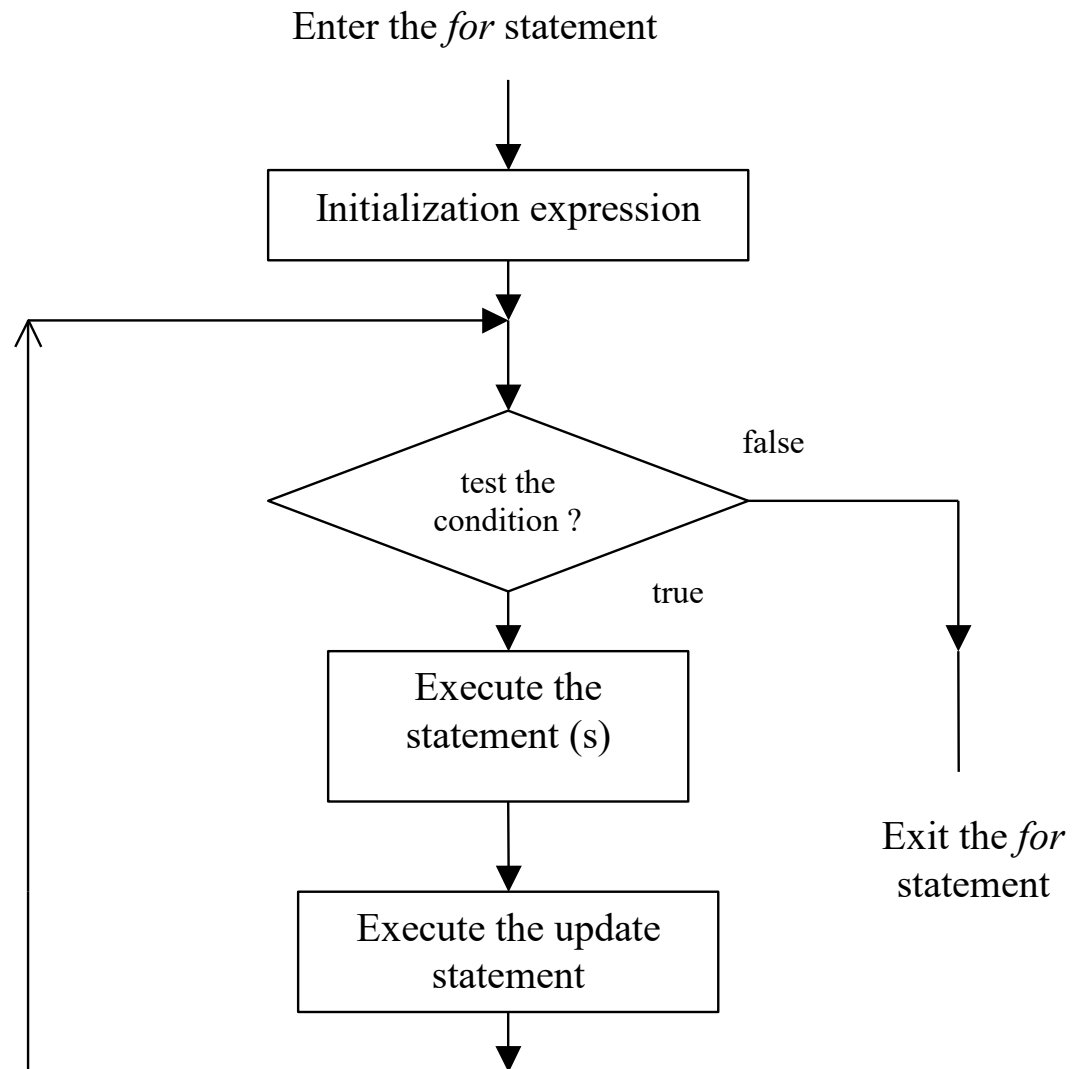
Example:

```
if (a > 20)
    goto esc;
else cout << a*a;
esc: cout << endl;
```

6. *for* LOOPS

- The *for* statement is used for repeating a statement or series of statements as long as a given conditional expression evaluates to true.
- One of the main differences between *while* statement and *for* statement is that in addition to a condition, you can also include code in the *for* statement
 - to initialize a counter variable and
 - changes its value with each iteration
- The syntax of the *for* statement:

```
for (initialization expression; condition; update statement) {  
    statement(s);  
}
```



Example 4.6.1

// This program prints the even number from 2 to 20

#include <iostream>

using namespace std;

int main() {

int count;

for (count = 2; count <= 20; count = count + 2)

cout << count << " ";

return 0;

}

The output of the above program:

2 4 6 8 12 14 16 18 20

Example 4.6.2 In this example, we solve this problem:

A person invests \$1000.00 in a saving account with 5 percent interest. Assuming that all interest is left on deposit in the account, calculate and print the amount of money in the account at the end of each year for 10 years. Use the following formula for determining these amounts:

$$a = p(1 + r)^n$$

where p is the original amount invested, r is the annual interest rate and n is the number of years and a is the amount on deposit at the end of the n th year.

```
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;
int main() {
    double amount = 0, principal = 1000.0, rate = 0.05;
    cout << "Year" << setw(21) << "Amount on deposit" << endl;
```

```
cout << setiosflags(ios::fixed | ios::showpoint) <<
    setprecision(2);
for (int year = 1; year <= 10; year++) {
    amount = principal*pow(1.0 + rate, year);
    cout << setw(4) << year << setw(21) << amount << endl;
}
return 0;
}
```

The output of the above program:

Year	Amount on deposit
1	1050.00
2	1102.50
3	1157.62
4	1215.51
5	1276.28
6	1340.10
7	1407.10
8	1477.46
9	1551.33
10	1628.89

7. NESTED LOOPS

In many situations, it is convenient to use a loop contained within another loop. Such loops are called *nested loops*.

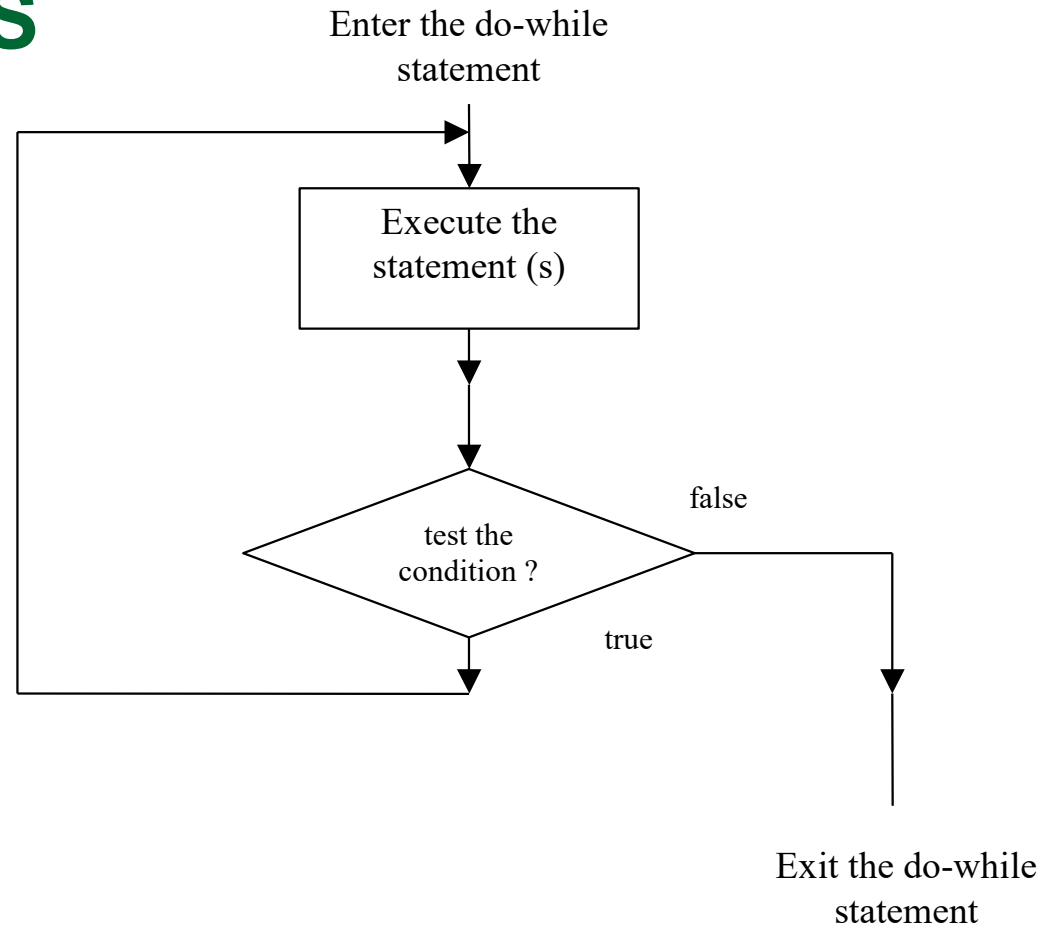
■ Example 4.7.1

```
#include <iostream>
using namespace std;
int main()
{
    const int MAXI = 5;
    const int MAXJ = 4;
    int i, j;
    for(i = 1; i <= MAXI; i++)    // start of outer loop
    {
        cout << "\n i is now " << i << endl;
        for(j = 1; j <= MAXJ; j++) // start of inner loop
            cout << " j = " << j;    // end of inner loop
    }                                // end of outer loop
    cout << endl;
    return 0;
}
```

The output of the program:

```
i is now 1
 j = 1 j = 2 j = 3 j = 4
i is now 2
 j = 1 j = 2 j = 3 j = 4
i is now 3
 j = 1 j = 2 j = 3 j = 4
i is now 4
 j = 1 j = 2 j = 3 j = 4
i is now 5
 j = 1 j = 2 j = 3 j = 4
```

8. do-while LOOPS



- ***do..while*** statement is used to create ***post-test*** loops.
- The syntax:
do {
 statements;
} while (conditional expression);

Example of *do-while*

- **Example 4.8.1** A program to find the sum of even numbers:
 $2+4+\dots+n$

```
#include<iostream>
using namespace std;
void main() {
    int max, sum = 0, digit;
    digit = 2;
    cout << " enter a number \n";
    cin >> max;
    do {
        sum = sum + digit;
        digit = digit + 2;
    } while (digit <= max);
    cout << " 2 + 4 +...+ "<< max << " sum = ";
    cout << sum << endl;
}
```

Output of the program:
Enter a number
10
2+4+...+10 sum = 30